Đề tài: LẬP TRÌNH GAME TIC TAC TOE TRÊN EMU8086

1.1: GIỚI THIỆU ĐỀ TÀI:

Tic Tac Toe (còn được gọi là Cờ ca-rô) là một trò chơi cổ điển mà hầu như ai cũng biết đến. Với luật chơi đơn giản nhưng đầy thách thức, nó đã trở thành trò chơi phổ biến trong nhiều năm qua.

Trong Tic Tac Toe, hai người chơi sẽ thay phiên nhau đặt dấu (X hoặc O) vào các ô trên bảng 3x3, với mục tiêu tạo ra một chuỗi ba dấu liên tiếp theo hàng ngang, hàng dọc hoặc đường chéo. Người chơi cần phải suy nghĩ chiến lược, xác định các nước đi quan trọng, và cố gắng ngăn chặn đối thủ của mình tạo ra chuỗi thắng. Trò chơi mang lại cảm giác thú vị và có thể trở thành một bài toán trí tuệ với chiến thuật và tư duy logic.

Emu8086 là một trình mô phỏng máy tính sử dụng vi xử lý Intel 8086. Nó cho phép lập trình và chạy các đoạn mã hợp ngữ (assembly) trong môi trường mô phỏng, giúp người học và nhà phát triển hiểu rõ hơn về cách hoạt động của vi xử lý và hệ thống máy tính thời kỳ đầu. Đề tài "Lập trình game Tic Tac Toe trên Emu8086" tập trung vào việc phát triển một phiên bản của trò chơi Tic Tac Toe trong môi trường Emu8086.

Với việc chọn đề tài này, chúng em mong muốn có thể:

- ***Rèn luyện kỹ năng lập trình với Assembly***: Lập trình Tic Tac Toe bằng Assembly yêu cầu sự hiểu biết sâu về ngôn ngữ này và cách tương tác với phần cứng máy tính. Đây là cơ hội tuyệt vời để phát triển kỹ năng lập trình và hiểu được cách mà máy tính hoạt động ở mức độ cơ bản.
- ***Xử lý tương tác và kiểm tra điều kiện***: Trong chế độ hai người chơi, lập trình viên phải xây dựng logic kiểm tra các nước đi và xác định người thắng cuộc. Điều này đòi hỏi sự cẩn thận và khả năng giải quyết vấn đề trong ngôn ngữ Assembly.
- ***Mang lại trải nghiệm cổ điển***: Lập trình Tic Tac Toe với chế độ hai người chơi tái hiện lại một trò chơi cổ điển mà mọi người yêu thích. Nó mang đến sự hồi hộp và tính cạnh tranh, đồng thời tạo ra một môi trường thú vị cho bạn bè và gia đình cùng chơi.

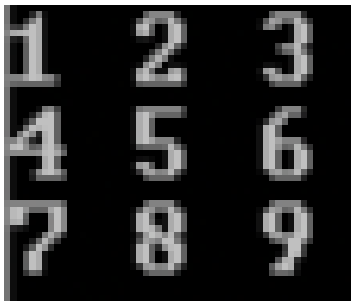
Dưới đây, chúng em (Hoàng Quý Phước, Đỗ Quốc Khánh, Võ Anh Quân, Trần Trọng Nghĩa), với sự phân công công việc của nhóm, xin phép được trình bày về đề tài: Lập trình game Tic Tac Toe trên Emu8086.

1.2: NỘI DUNG CHÍNH CỦA ĐỀ TÀI:

1. **Nội dung chính:** Lập trình game Tic Tac Toe bằng Assembly trên Emu8086
2. **Tổng quan về trò chơi:**
    a. Tic Tac Toe là trò chơi dành cho 2 người chơi, với bàn cờ kích thước 3x3. Mỗi người chơi lần lượt đặt dấu hiệu của mình, một người chọn "X" và người còn lại chọn "O".
    b. Mục tiêu của trò chơi là tạo ra một chuỗi ba dấu hiệu liên tiếp theo hàng ngang, hàng dọc hoặc đường chéo. Người chơi đầu tiên đạt được mục tiêu này sẽ là người chiến thắng. Nếu điền hết bàn cờ mà không tìm được ra người chiến thắng sẽ có kết quả là hòa.
    c. Cách chơi:
        i. Người chơi thứ nhất sẽ bắt đầu bằng cách đặt dấu "X" vào một ô bất kỳ trên bàn cờ. Người chơi thứ hai sẽ tiếp tục với dấu "O".
        ii. Người chơi thay phiên nhau đặt dấu hiệu của mình cho đến khi một người chiến thắng hoặc bàn cờ đầy mà không có ai thắng, dẫn đến kết quả hòa.
        iii. Người chơi cần suy nghĩ chiến lược và tìm cách ngăn chặn đối thủ trong khi cố gắng tạo ra chuỗi ba dấu hiệu liên tiếp.
        iv. Các chức năng có trong trò chơi:
            1. Tạo ra bàn cờ 3x3 cho phép người chơi chọn số từ 1 đến 9 tương ứng với các ô trên bảng, điền ‘X’ hoặc ‘O’ vào ô đó.
            2. Nếu có người chiến thắng (xuất hiện 1 chuỗi 3 dấu XXX hoặc OOO liên tiếp theo hàng ngang, hàng dọc hoặc đường chéo), sẽ in ra thông báo người chiến thắng.
            3. Nếu bàn cờ đầy mà không có ai chiến thắng, thông báo cho biết kết quả là hòa.
            4. Đặc điểm của trò chơi:
                a. Trò chơi được thiết kế dành cho 2 người.
                b. Giao diện trò chơi đơn giản.
                c. Trò chơi kinh điển, mang tính cạnh tranh cao.

3. Giải thuật chính của chương trình & Lời giải lựa chọn

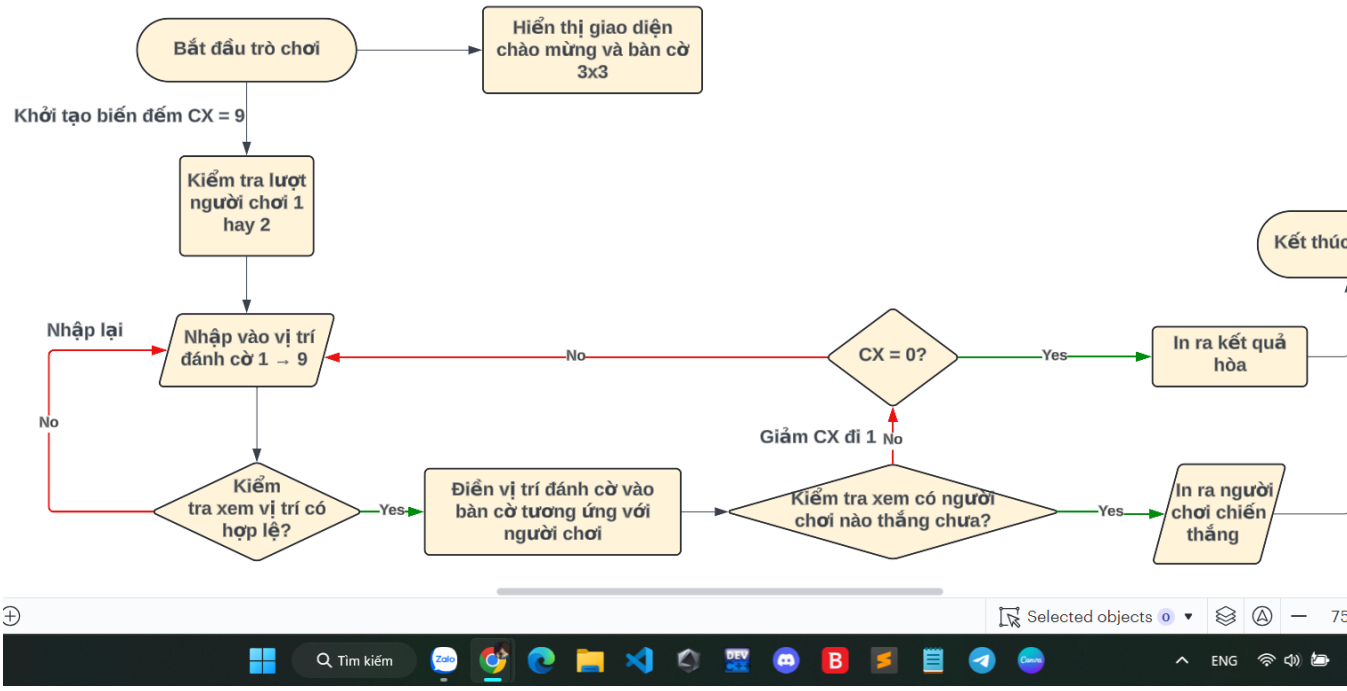
- **Cấu trúc chính của chương trình:** Sử dụng một mảng 1 chiều gồm 9 phần tử (MANG), tương ứng với các vị trí trên bảng Tic Tac Toe. Mỗi phần tử ban đầu được gán một ký hiệu đại diện cho vị trí tương ứng (từ '1' đến '9').



- **Thuật toán chương trình:**
- ***Xác định lượt chơi***: Sử dụng biến **CX** để đếm số lượt chơi, bắt đầu từ 9 (tối đa 9 lượt). Biến **PLAYER** được sử dụng để xác định lượt chơi của ai, với "X" cho lượt lẻ và "O" cho lượt chẵn.
- ***Nhập dữ liệu từ người chơi***: Thông qua việc đọc từ bàn phím, người chơi nhập vị trí muốn đánh dấu. Nếu dữ liệu không hợp lệ, yêu cầu người chơi nhập lại.
- ***Cập nhật bảng Tic Tac Toe***: Sau mỗi lượt chơi, trò chơi cập nhật vị trí đã nhập với ký hiệu của người chơi tương ứng.
- ***Kiểm tra chiến thắng***: Sau mỗi lượt chơi, trò chơi kiểm tra các điều kiện chiến thắng. Có tổng cộng 8 cách để thắng (3 hàng ngang, 3 hàng dọc, 2 hàng chéo). Nếu người chơi đạt được một trong các điều kiện này, trò chơi thông báo người thắng cuộc.
- ***Xử lý kết quả***: Nếu một người chơi thắng, trò chơi dừng lại và thông báo người chiến thắng. Nếu không có người thắng sau 9 lượt chơi, trò chơi kết luận là hòa.

1.3: MIÊU TẢ CHƯƠNG TRÌNH:

Lưu đồ thuật toán (Flowcharts) của chương trình:

- 

➡ **Dữ liệu đầu vào:** 2 Người chơi lần lượt nhập vào ô chọn (1 đến 9).

➡ **Dữ liệu đầu ra:** In ra kết quả (Người thắng hoặc Cả 2 hòa).

Phân tích chương trình: https://ideone.com/n5CU9L

```
Name "Tic Tac Toe" ; Đặt tên chương trình"Tic Tac Toe"
Org 100h           ; Thiết lập vị trí bắt đầu trong bộ nhớ
.DATA              ; Khối dữ liệu
        MANG DB '1','2','3'     ; Khai báo mảng 2D grid
             DB '4','5','6'
             DB '7','8','9'
        PLAYER DB ?                                          ; Khai báo biến cho người chơi
        WELCOME DB 'Welcome to Tic Tac Toe! $'               ; Thông điệp chào mừng
        INPUT DB 'Enter Position Number, PLAYER Turn is: $'  ; Thông điệp nhập dữ liệu
        DRAW DB 'DRAW! $'                                    ; thông điệp hòa
        WIN DB 'PLAYER WIN: $'                               ; thông điệp chiến thắng
```

- **Name** "Tic Tac Toe": Đặt tên chương trình là Tic Tac Toe
- **Org** 100H: Thiết lập vị trí bắt đầu của mã lệnh tại địa chỉ bộ nhớ 100h
- **.Data**: Đánh dấu bắt đầu của phân đoạn dữ liệu, nơi lưu trữ dữ liệu tĩnh.
- **MANG**: Khai báo mảng 3x3 đại diện cho bảng Tic Tac Toe với các giá trị ban đầu là các số từ 1 đến 9, để người chơi có thể chọn.
- **PLAYER**: Khai báo biến để xác định người chơi hiện tại.
- **WELCOME**: Thông điệp chào mừng

- **INPUT**: Thông điệp nhập dữ liệu
- **DRAW**: Thông điệp hoà
- **WIN**: Thông điệp chiến thắng

```
.CODE    ; Khối mã lệnh
main:
        mov cx,9                ; Lặp 9 lần vì số lượng tối đa các lượt chơi là 9
x:
        call XOA_MAN_HINH       ; Xóa màn hình để cho giao diện đẹp hơn
        call PRINT_WELCOME      ; In thông điệp chào mừng
        call PRINT_MANG         ; In bảng lưới
        mov bx, cx              ; Di chuyển cx vào bx
        and bx, 1               ; Kiểm tra số chẵn hoặc lẻ
        cmp bx, 0               ; So sánh kết quả AND
        je isEven               ; Nhảy đến isEven nếu kết quả 0 (chẵn)
        mov PLAYER, 'x'         ; Nếu là số lẻ thì là lượt của người chơi x
        jmp endif               ; Chuyển đến bước tiếp theo
isEven:
        mov PLAYER, 'o'         ; Nếu là số chẵn thì là lượt của người chơi o
endif:
```

- **main**: Điểm bắt đầu của chương trình.
- **mov cx,9**: Khởi tạo số lượt chơi, mỗi lượt là một ô trên bảng Tic Tac Toe.
- **x**: Nhãn cho vòng lặp chính của trò chơi.
- **call XOA_MAN_HINH**: Gọi hàm để xóa màn hình trước mỗi lượt chơi.
- **call PRINT_WELCOME**: In thông điệp chào mừng.
- **call PRINT_MANG**: In ra bảng lưới trò chơi.
- **mov bx, cx; and bx, 1; cmp bx, 0, je isEven**: Xác định người chơi hiện tại là 'x' hoặc 'o' dựa trên số lượt chơi đã thực hiện.

```
endif:
  NOT_VALID:
        call IN_DONG_MOI        ; In dòng mới
        call IN_NHAP            ; In thông điệp nhập liệu
        call NHAP               ; Đọc dữ liệu đầu vào, al chứa vị trí trên bảng lưới

        push cx                 ; Đẩy cx vào ngăn xếp
        mov cx, 9               ; Thiết lập số lượng vòng lặp
        mov bx, 0               ; Chỉ số để truy cập bảng lưới
y:
        cmp MANG[bx], al        ; Kiểm tra vị trí trên bảng lưới với dữ liệu đầu vào
        je UPDATE               ; Nếu trùng khớp cập nhật vị trí của người chơi(x hoặc o)
        jmp CONTINUE            ; Tiếp tục nếu không trùng
UPDATE:
        mov dl, PLAYER          ; Di chuyển ng chơi vào dl
        mov MANG[bx], dl        ; Cập nhật bảng lưới với người chơi
CONTINUE:
        inc bx                  ; Tăng chỉ số
        loop y                  ; Lặp đến khi hoàn tất
        pop cx                  ; Lấy giá trị cx ra khỏi ngăn xếp
        call CHECKWIN           ; Kiểm tra kết quả chơi
        loop x                  ; Lặp lại chương trình
        call PRINT_DRAW         ; nếu không ai thắng in hòa
programEnd:
        mov     ah, 0           ; Thiết lập thanh ghi AH thành giá trị 0
        int     16h
ret
```

- **NOT_VALID**: Nhãn để xử lý trường hợp người chơi đã chọn một ô không hợp lệ.
- **call IN_DONG_MOI**: In một dòng trống để tạo khoảng trống giữa các lượt chơi.
- **call IN_NHAP**: In thông điệp yêu cầu người chơi nhập vào một ô trên bảng.
- **call NHAP**: Đọc giá trị đầu vào từ người chơi, và lưu giá trị này vào thanh ghi al.
- **push cx**: Lưu trữ giá trị của thanh ghi cx vào ngăn xếp.
- **mov cx, 9**: Khởi tạo thanh ghi cx với giá trị 9, đại diện cho số lượt chơi còn lại.
- **mov bx, 0**: Khởi tạo thanh ghi bx với giá trị 0, sẽ được sử dụng như một chỉ số để truy cập vào lưới.
- **y**: Nhãn cho vòng lặp trong quá trình tìm kiếm vị trí trên lưới.
- **cmp MANG[bx], al**: So sánh giá trị trong ô lưới với giá trị mà người chơi đã chọn.
- **je UPDATE**: Nếu giá trị trong ô lưới bằng với giá trị mà người chơi đã chọn, chương trình sẽ nhảy đến nhãn update.
- **jmp CONTINUE**: Nếu không, chương trình sẽ nhảy đến nhãn continue.
- **UPDATE**: Nhãn để cập nhật giá trị của ô lưới với biểu tượng của người chơi hiện tại ('x' hoặc 'o').
- **mov dl, PLAYER**: Di chuyển biểu tượng của người chơi hiện tại vào thanh ghi dl vì không thể di chuyển trực tiếp từ bộ nhớ này sang bộ nhớ khác.
- **mov MANG[bx], dl**: Cập nhật giá trị của ô lưới với biểu tượng của người chơi hiện tại.
- **CONTINUE**: Nhãn để tiếp tục vòng lặp.

- **inc bx**: Tăng chỉ số bx lên 1 để di chuyển đến ô lưới tiếp theo.
- **loop y**: Lặp lại vòng lặp y cho đến khi cx (số lượt chơi còn lại) giảm về 0.
- **pop cx**: Lấy giá trị của thanh ghi cx từ ngăn xếp.
- **call CHECKWIN**: Kiểm tra xem đã có người chiến thắng chưa.
- **loop x**: Lặp lại vòng lặp x cho đến khi cx giảm về 0, tức là đã chơi hết 9 lượt mà không có người chiến thắng.
- **call PRINT_DRAW**: In ra thông báo hòa nếu không có người chiến thắng sau khi chơi hết lượt.
- **programEnd**: Nhãn cho kết thúc chương trình.

```
; Các hàm
PRINT_MANG:                             ; Thủ tục in bảng lưới
        push cx                         ; Đẩy cx vào ngăn xếp
        mov bx,0                        ; Thiết lập chỉ số ban đầu
        mov cx,3                        ; Số dòng của bảng lưới
        x1:
                call IN_DONG_MOI        ; In dòng mới
                push cx                 ; Đẩy cx vào ngăn xếp
                mov cx, 3               ; Số cột của bảng lưới
        x2:
        mov dl, MANG[bx]                ; Di chuyển giá trị grid vào dl
        mov ah, 2h                      ; Câu lệnh in ký tự
        int 21h
        call PRINT_Space                ; Gọi hàm in khoảng trắng
        inc bx                          ; Tăng chỉ số
        loop x2                         ; Lặp lại qua các cột
        pop cx                          ; Lấy lại giá trị cx
        loop x1                         ; Lap lai qua cac dong
        pop cx                          ; Lấy lại giá trị cx
        call IN_DONG_MOI                ; In dòng mới
ret
```

- **PRINT_MANG:** Nhãn để xử lý in bảng lưới.
- **push cx và pop cx**: Trước và sau khi gọi hàm PRINT_MANG, giá trị của thanh ghi cx được đẩy vào và lấy ra từ ngăn xếp (stack). Điều này được thực hiện để bảo vệ giá trị của thanh ghi cx khi hàm được gọi nhiều lần, bởi vì hàm này sử dụng thanh ghi cx trong các vòng lặp.
- **Vòng lặp chính (x1)**: Vòng lặp này đi qua các hàng của lưới Tic Tac Toe. Trong mỗi vòng lặp, hàm **IN_DONG_MOI** được gọi để in một dòng mới trước khi in ra các ký tự của hàng tiếp theo. Giá trị của cx (số lần lặp) được đẩy vào ngăn xếp để bảo vệ giá trị này trong vòng lặp bên trong.

- **Vòng lặp lồng nhau (x2)**: Mỗi lần lặp qua hàng (x1), chúng ta lặp lại qua các ô của hàng đó (x2). Trong mỗi ô, ký tự tương ứng từ mảng **MANG** được lấy ra và in ra màn hình. Ký tự này được chuyển đổi từ mã ASCII sang ký tự bằng cách trừ đi 30h. Sau đó, một dấu cách được in để phân tách giữa các ô.
- **pop cx**: Sau khi kết thúc vòng lặp lồng nhau (x2), giá trị cx được lấy ra từ ngăn xếp để khôi phục giá trị trước khi vòng lặp (x1) bắt đầu.
- **call IN_DONG_MOI** cuối cùng: Sau khi hoàn thành tất cả các hàng của lưới, hàm printNewLine được gọi để in một dòng mới để tạo ra một khoảng trống giữa các lần in lưới.
- **ret**: Cuối cùng, hàm trả về (ret). Điều này kết thúc hàm PRINT_MANG và chương trình tiếp tục thực thi từ điểm mà hàm được gọi.

```
IN_DONG_MOI:                    ; Thủ tục in dòng mới
        mov dl, 0ah             ; Kí tự xuống dòng
        mov ah, 2               ; Câu lệnh in kí tự
        int 21h                 ; Gọi ngắt để in kí tự
        mov dl, 13
        mov ah, 2               ; Câu lệnh in kí tự
        int 21h                 ; Gọi ngắt để in kí tự
ret
```

- **mov ah, 2**: Đặt giá trị của thanh ghi AH thành 2, cho biết rằng chúng ta muốn thực hiện một hàm ngắt liên quan đến việc in ký tự.
- **int 21h**: Gọi ngắt để thực hiện hàm in ký tự, trong trường hợp này là kí tự xuống dòng.
- **mov dl, 13**: Đặt giá trị của thanh ghi DL thành 13, đây là mã ASCII của kí tự carriage return.
- **mov ah, 2**: Đặt giá trị của thanh ghi AH thành 2, để thực hiện hàm ngắt in ký tự.
- **int 21h**: Gọi ngắt để thực hiện hàm in ký tự, trong trường hợp này là ký tự carriage return.
- **ret**: Cuối cùng, hàm trả về (ret), kết thúc thủ tục IN_DONG_MOI và chương trình tiếp tục thực thi từ điểm mà thủ tục được gọi.

```
PRINT_Space:                        ; Thủ tục in khoảng trắng
        mov dl, 32                  ; Mã ascii của khoảng trắng
        mov ah, 2                   ; Câu lệnh in kí tự
        int 21h                     ; Gọi ngắt để in kí tự
ret
```

- **mov dl, 32**: Đặt giá trị của thanh ghi DL thành 32, đây là mã ASCII của ký tự khoảng trắng.
- **mov ah, 2**: Đặt giá trị của thanh ghi AH thành 2, cho biết rằng chúng ta muốn thực hiện một hàm ngắt liên quan đến việc in ký tự.
- **int 21h**: Gọi ngắt để thực hiện hàm in ký tự, trong trường hợp này là ký tự khoảng trắng.
- **ret**: Cuối cùng, hàm trả về (ret), kết thúc thủ tục PRINT_Space và chương trình tiếp tục thực thi từ điểm mà thủ tục được gọi.

```
NHAP:   ; thủ tục đọc dữ liệu đầu vào

        mov ah, 1               ; cho phép nhập kí tự
        int 21h                 ; gọi ngắt để nhập dữ liệu
        cmp al, '1'             ; kiểm tra giá trị nhập vào
        je VALID
        cmp al, '2'
        je VALID
        cmp al, '3'
        je VALID
        cmp al, '4'
        je VALID
        cmp al, '5'
        je VALID
        cmp al, '6'
        je VALID
        cmp al, '7'
        je VALID
        cmp al, '8'
        je VALID
        cmp al, '9'
        je VALID
        jmp NOT_VALID          ; quay lại vị trí không hợp lệ
        VALID:                 ; điểm hợp lệ
ret
```

- **NHAP:** Thủ tục đọc dữ liệu đầu vào.
- **mov ah, 1:** Cho phép nhập kí tự.

- **int 21h:** Gọi ngắt để nhập dữ liệu.
- **cmp al, '1':** Nếu al = 1 nhảy đến VALID.
- **je VALID :** Chuyển đến nhãn VALID.
- **cmp al, '2':** Nếu al = 2 nhảy đến VALID.
- **je VALID :** Chuyển đến nhãn VALID.
- **cmp al, '3':** Nếu al = 3 nhảy đến VALID.
- **je VALID:** Chuyển đến nhãn VALID.
- **cmp al, '4':** Nếu al = 4 nhảy đến VALID.
- **je VALID:** Chuyển đến nhãn VALID.
- **cmp al, '5':** Nếu al = 5 nhảy đến VALID.
- **je VALID:** Chuyển đến nhãn VALID.
- **cmp al, '6':** Nếu al = 6 nhảy đến VALID.
- **je VALID:** Chuyển đến nhãn VALID.
- **cmp al, '7':** Nếu al = 7 nhảy đến VALID.
- **je VALID:** Chuyển đến nhãn VALID.
- **cmp al, '8':** Nếu al = 8 nhảy đến VALID.
- **je VALID:** Chuyển đến nhãn VALID.
- **cmp al, '9':** Nếu al = 9 nhảy đến VALID.
- **je VALID:** Chuyển đến nhãn VALID.
- **je NOT_VALID**: Quay lại vị trí không hợp lệ.
- **VALID:** Điểm hợp lệ.
- **ret**

```
PRINT_WELCOME:                          ; Thủ tục in thông điệp chào mừng
        lea dx, WELCOME                 ; Tải địa chỉ của thông điệp vào dx
        mov ah, 9                       ; Câu lệnh in chuỗi
        int 21h                         ; Gọi ngắt để in chuỗi
ret
```

- **PRINT_WELCOME:** Thủ tục in thông điệp chào mừng.
- **lea dx, WELCOME:** Tải địa chỉ của thông điệp vào DX.
- **mov ah, 9**: Câu lệnh in chuỗi.
- **int 21h:** Gọi ngắt để in chuỗi.
- **ret**

```
PRINT_DRAW:                                 ; Thủ tục in thông điệp hoa
        call IN_DONG_MOI                    ; In dòng mới
        lea dx, DRAW                        ; Tải địa chỉ thông điệp
        mov ah, 9                           ; Câu lệnh in chuỗi
        int 21h                             ; Gọi ngắt để in chuỗi
ret
```

- **CALL IN_DONG_MOI**: Gọi thủ tục IN_DONG_MOI để in ra một dòng mới trước khi in thông điệp hoà. Điều này giúp tạo ra một khoảng trống giữa các thông điệp trên màn hình.
- **lea dx, DRAW**: Tải địa chỉ của chuỗi thông điệp "DRAW" vào thanh ghi DX. Chuỗi này sẽ được in ra màn hình để thông báo rằng trò chơi kết thúc với kết quả là hòa.
- **mov ah, 9**: Đặt giá trị của thanh ghi AH thành 9, cho biết rằng chúng ta muốn thực hiện hàm ngắt liên quan đến việc in chuỗi ký tự.
- **int 21h**: Gọi ngắt để thực hiện hàm in chuỗi ký tự. Trong trường hợp này, thông điệp "DRAW" sẽ được in ra màn hình.
- **ret**: Cuối cùng, hàm trả về (ret), kết thúc thủ tục PRINT_DRAW và chương trình tiếp tục thực thi từ điểm mà thủ tục được gọi.

```
PRINT_WIN:                                  ; Thủ tục in thông điệp chiến thắng
        call IN_DONG_MOI                    ; In dòng mới
        call PRINT_MANG                     ; In bảng lưới lần cuối
        lea dx, WIN                         ; Tải địa chỉ thông điệp
        mov ah, 9                           ; Câu lệnh in chuỗi
        int 21h                             ; Gọi ngắt để in chuỗi
        mov dl, PLAYER                      ; Di chuyển giá trị người chơi vào dl
        mov ah, 2h                          ; Câu lệnh in kí tự
        int 21h                             ; Gọi ngắt để in kí tự
        jmp programEnd                      ; Quay lại kết thúc chương trình
ret
```

- **call PRINT_MANG**: Gọi hàm PRINT_MANG để in ra bảng lưới một lần nữa trước khi in thông điệp chiến thắng. Điều này đảm bảo rằng bảng lưới sẽ được hiển thị đầy đủ cho người chơi trước khi thông điệp chiến thắng xuất hiện.
- **lea dx, WIN**: Tải địa chỉ của chuỗi thông điệp "WIN" vào thanh ghi DX. Chuỗi này sẽ được in ra màn hình để thông báo rằng một người chơi đã chiến thắng.
- **mov dl, PLAYER**: Di chuyển giá trị của biến PLAYER vào thanh ghi DL. Giá trị này đại diện cho người chơi đã chiến thắng.

- **mov ah, 2h**: Đặt giá trị của thanh ghi AH thành 2h, cho biết rằng chúng ta muốn thực hiện hàm ngắt liên quan đến việc in ký tự.
- **int 21h**: Gọi ngắt để thực hiện hàm in ký tự. Trong trường hợp này, ký tự đại diện cho người chiến thắng sẽ được in ra màn hình.
- **jmp programEnd**: Chương trình sẽ chuyển đến nhãn programEnd để kết thúc chương trình sau khi thông điệp chiến thắng được hiển thị.
- **ret**: Cuối cùng, hàm trả về (ret), kết thúc thủ tục PRINT_WIN.

```
IN_NHAP:                                ; Thủ tục in thông điệp nhập liệu
        lea dx, INPUT                   ; Tải địa chỉ thông điệp
        mov ah, 9                       ; Câu lệnh in chuỗi
        int 21h                         ; Gọi ngắt để in chuỗi
        mov dl, PLAYER                  ; Di chuyển giá trị người chơi vào dl
        mov ah, 2h                      ; Câu lệnh in kí tự
        int 21h                         ; Gọi ngắt để in kí tự
        call PRINT_Space                ; Gọi thủ tục in khoảng trắng
ret
```

- **lea dx, INPUT**: Tải địa chỉ của chuỗi thông điệp "INPUT" vào thanh ghi dx. Chuỗi này chứa thông điệp yêu cầu người chơi nhập liệu.
- **mov ah, 9**: Đặt giá trị của thanh ghi ah thành 9, cho biết rằng chúng ta muốn thực hiện hàm ngắt liên quan đến việc in chuỗi ký tự.
- **int 21h**: Gọi ngắt để thực hiện hàm in chuỗi ký tự. Trong trường hợp này, thông điệp "INPUT" sẽ được hiển thị trên màn hình.
- **mov dl, PLAYER**: Di chuyển giá trị của biến PLAYER vào thanh ghi dl. Điều này sẽ chứa ký tự biểu tượng của người chơi hiện tại.
- **mov ah, 2h**: Đặt giá trị của thanh ghi ah thành 2h, cho biết rằng chúng ta muốn thực hiện hàm ngắt liên quan đến việc in ký tự.
- **int 21h**: Gọi ngắt để thực hiện hàm in ký tự. Trong trường hợp này, ký tự biểu tượng của người chơi sẽ được hiển thị trên màn hình.
- **call PRINT_Space**: Gọi thủ tục PRINT_Space để in ra một khoảng trắng sau khi hiển thị thông điệp và biểu tượng của người chơi.
- **ret**: Cuối cùng, hàm trả về (ret), kết thúc thủ tục IN_NHAP.

```
CHECKWIN:                           ; Thủ tục kiểm tra kết quả
        mov bl, MANG[0]             ; Kiểm tra hàng 0
        cmp bl, MANG[1]             ; So sánh giá trị đầu tiên và thứ 2
        jne skip1                   ; Nếu không giống nhau bỏ qua
        cmp bl, MANG[2]             ; So sánh giá trị đầu tiên và thứ 3
        jne skip1                   ; Nếu không giống nhau bỏ qua
        call PRINT_WIN              ; Nếu giống nhau in thông điệp chiến thắng
```

- **call PRINT_WIN**: Nếu các giá trị trong hàng 0 của mảng MANG là giống nhau (tức là cùng một biểu tượng của một người chơi), thì hàm sẽ gọi thủ tục PRINT_WIN để in thông điệp chiến thắng lên màn hình.
- **skip1**: Nhãn skip1 được sử dụng để bỏ qua các bước kiểm tra tiếp theo nếu các giá trị trong hàng 0 không giống nhau.
- Việc kiểm tra chiến thắng tiếp theo sẽ thực hiện cho các hàng còn lại và các đường chéo của bảng lưới Tic Tac Toe. Điều này đảm bảo rằng mọi phương án chiến thắng đều được kiểm tra và xác định.

```
skip1:                              ; Vị trí bỏ qua
        mov bl, MANG[3]             ; Kiểm tra hàng 1
        cmp bl, MANG[4]             ; So sánh giá trị đầu tiên và thứ 2 của hàng 1
        jne skip2                   ; Nếu không giống nhau bỏ qua
        cmp bl, MANG[5]             ; So sánh giá trị đầu tiên và thứ 3 của hàng 1
        jne skip2                   ; Nếu không giống nhau bỏ qua
        call PRINT_WIN              ; Nếu giống nhau in thông điệp chiến thắng
```

- **skip1:** Vị trí bỏ qua.
- **mov bl, MANG[3]:** Kiểm tra hàng 1.
- **cmp bl, MANG[4]:** So sánh giá trị đầu tiên và thứ 2 của hàng 1.
- **jne skip2:** Nếu không giống nhau thì đến nhãn skip2.
- **cmp bl, MANG[5]:** So sánh giá trị đầu tiên và thứ 3 của hàng 1.
- **jne skip2:** Nếu không giống nhau thì đến nhãn skip2.
- **call PRINT_WIN:** Nếu giống nhau gọi hàm in thông điệp chiến thắng.

```
skip2:                          ; Vị trí bỏ qua
        mov bl, MANG[6]         ; Kiểm tra hàng 2
        cmp bl, MANG[7]         ; So sánh giá trị đầu tiên và thứ 2 của hàng 2
        jne skip3               ; Nếu không giống nhau bỏ qua
        cmp bl, MANG[8]         ; So sánh giá trị đầu tiên và thứ 3 của hàng 2
        jne skip3               ; Nếu không giống nhau bỏ qua
        call PRINT_WIN          ; Nếu giống nhau in thông điệp chiến thắng
```

- **skip2:** Vị trí bỏ qua.
- **mov bl, MANG[6]**: Kiểm tra hàng 2.
- **cmp bl, MANG[7]:** So sánh giá trị đầu tiên và thứ 2 của hàng 2.
- **ine skip3:** Nếu không giống nhau thì đến nhãn skip3.
- **cmp bl, MANG[8]:** So sánh giá trị đầu tiên và thứ 3 của hàng 2.
- **jne skip3:** Nếu không giống nhau thì đến nhãn skip3.
- **call PRINT_WIN:** Nếu giống nhau gọi hàm in thông điệp chiến thắng.
- **skip3:** Vị trí bỏ qua.

```
skip3:                          ; Vị trí bỏ qua
        mov bl, MANG[0]         ; Kiểm tra cột 0
        cmp bl, MANG[3]         ; So sánh vị trí đầu tiên và thứ 2 của cột 0
        jne skip4               ; Nếu không giống nhau bỏ qua
        cmp bl, MANG[6]         ; So sánh vị trí đầu tiên và thứ 3 của cột 0
        jne skip4               ; Nếu không giống nhau bỏ qua
        call PRINT_WIN          ; Nếu giống nhau in thông điệp chiến thắng
```

- **mov bl, MANG[0]:** Kiểm tra cột 0.
- **cmp bl, MANG[3]:** So sánh vị trí đầu tiên và thứ 2 của cột 0.
- **jne skip4:** Nếu không giống nhau thì đến nhãn skip4.
- **cmp bl, MANG[6]:** So sánh vị trí đầu tiên và thứ 3 của cột 0.
- **jne skip4:** Nếu không giống nhau thì đến nhãn skip4.
- **call PRINT_WIN:** Nếu giống nhau gọi hàm in thông điệp chiến thắng.

```
skip4:                          ; Vị trí bỏ qua
        mov bl, MANG[1]         ; Kiểm tra cột 1
        cmp bl, MANG[4]         ; So sánh vị trí đầu tiên và thứ 2 của cột 1
        jne skip5               ; Nếu không giống nhau bỏ qua
        cmp bl, MANG[7]         ; So sánh vị trí đầu tiên và thứ 3 của cột 1
        jne skip5               ; Nếu không giống nhau bỏ qua
        call PRINT_WIN          ; Nếu giống nhau in thông điệp chiến thắng
```

- **skip4:** Vị trí bỏ qua.
- **mov bl, MANG[1]:** Kiểm tra cột 1.
- **cmp bl, MANG[4]:** So sánh vị trí đầu tiên và thứ 2 của cột 1.
- **jne skip5**: Nếu không giống nhau thì đến nhãn skip5.
- **cmp bl, MANG[7]:** So sánh vị trí đầu tiên và thứ 3 của cột 1.
- **jne skip5:** Nếu không giống nhau thì đến nhãn skip5.
- **call PRINT_WIN:** Nếu giống nhau gọi hàm in thông điệp chiến thắng.

```
skip5:                          ; Vị trí bỏ qua
        mov bl, MANG[2]         ; Kiểm tra cột 2
        cmp bl, MANG[5]         ; So sánh vị trí đầu tiên và thứ 2 của cột 2
        jne skip6               ; Nếu không giống nhau bỏ qua
        cmp bl, MANG[8]         ; So sánh vị trí đầu tiên và thứ 3 của cột 2
        jne skip6               ; Nếu không giống nhau bỏ qua
        call PRINT_WIN          ; Nếu giống nhau in thông điệp chiến thắng
```

- **skip5:** Vị trí bỏ qua .
- **mov bl, MANG[2]:** Kiểm tra cột 2.
- **cmp bl, MANG[5]:** So sánh vị trí đầu tiên và thứ 2 của cột 2.
- **jne skip6:** Nếu không giống nhau thì đến nhãn skip6.
- **cmp bl, MANG[8]:** So sánh vị trí đầu tiên và thứ 3 của cột 2.
- **jne skip6:** Nếu không giống nhau thì đến nhãn skip6.
- **call PRINT_WIN:** Nếu giống nhau gọi hàm in thông điệp chiến thắng.

```
skip6:                          ; vị trí bỏ qua

            mov bl, MANG[0]     ; Kiểm tra đường chéo chính
            cmp bl, MANG[4]     ; So sánh giá trị đầu tiên và thứ 2 của đường chéo chính
            jne skip7           ; Nếu không giống nhau bỏ qua
            cmp bl, MANG[8]     ; So sánh giá trị đầu tiên và thứ 3 của đường chéo chính
            jne skip7           ; Nếu không giống nhau bỏ qua
            call PRINT_WIN      ; Nếu giống nhau in thông điệp chiến thắng
```

- **skip6:** Vị trí bỏ qua.
- **mov bl, MANG[0]:** Kiểm tra đường chéo chính.
- **cmp bl, MANG[4]:** So sánh giá trị đầu tiên và thứ 2 của đường chéo chính.
- **jne skip7:** Nếu không giống nhau thì đến nhãn skip7.
- **cmp bl, MANG[8]**: So sánh giá trị đầu tiên và thứ 3 của đường chéo chính.

- **jne skip7**: Nếu không giống nhau thì đến nhãn skip7.
- **call PRINT_WIN:** Nếu giống nhau gọi hàm in thông điệp chiến thắng.

```
skip7:                          ; vị trí bỏ qua
        mov bl, MANG[2]         ; Kiểm tra đường chéo phụ
        cmp bl, MANG[4]         ; So sánh giá trị đầu tiên và thứ 2 của đường chéo phụ
        jne skip8               ; Nếu không giống nhau bỏ qua
        cmp bl, MANG[6]         ; So sánh giá trị đầu tiên và thứ 3 của đường chéo phụ
        jne skip8               ; Nếu không giống nhau bỏ qua
        call PRINT_WIN          ; Nếu giống nhau in thông điệp chiến thắng

skip8:                          ; Vị trí bỏ qua
ret
```

- **skip7:** Vị trí bỏ qua.
- **cmp bl, MANG[2]**: Kiểm tra đường chéo phụ.
- **cmp bl, MANG[4]:** So sánh giá trị đầu tiên và thứ 2 của đường chéo phụ.
- **jne skip8**: Nếu không giống nhau thì đến nhãn skip8.
- **cmp bl, MANG[6]:** So sánh giá trị đầu tiên và thứ 3 của đường chéo phụ.
- **jne skip8:** Nếu không giống nhau thì đến nhãn skip8.
- **call PRINT_WIN**: Nếu giống nhau gọi hàm in thông điệp chiến thắng.
- **skip8:** Vị trí bỏ qua.
- **ret**

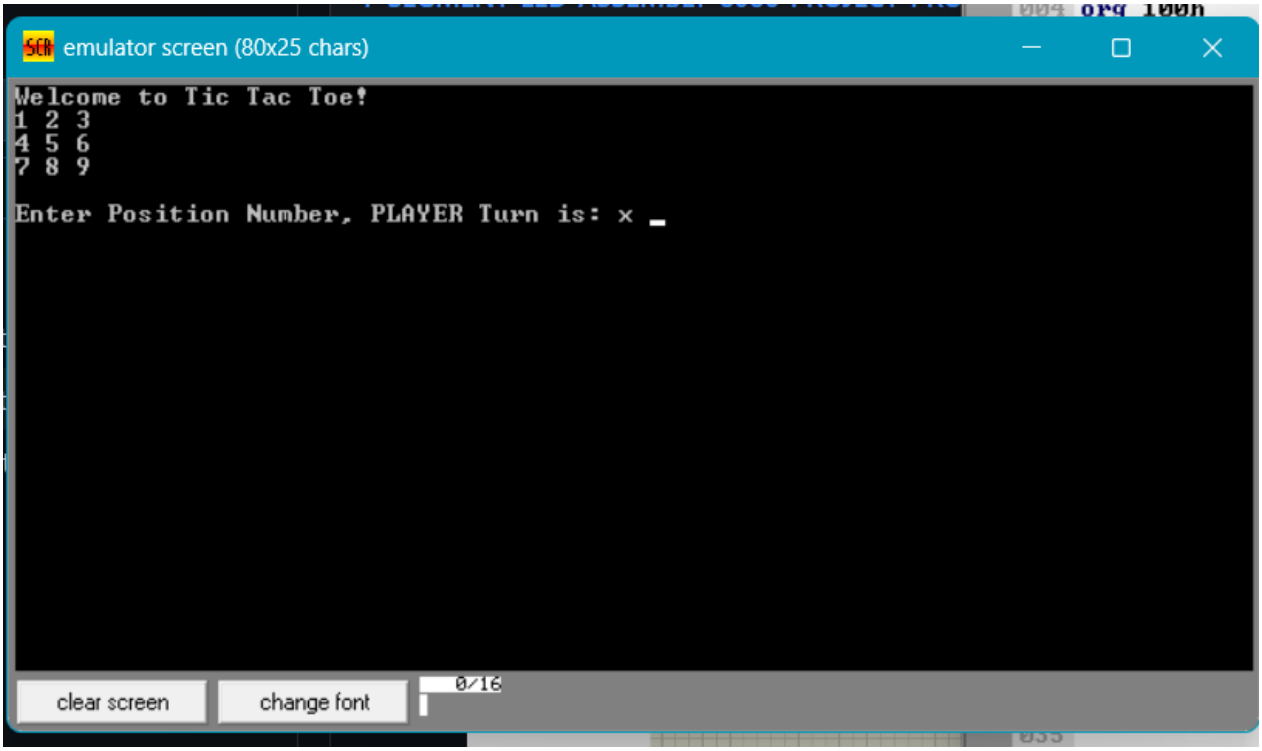
```
XOA_MAN_HINH:                   ; Thủ tục xóa màn hình
        mov ax, 3               ; Xóa màn hình
        int 10h                 ; Gọi ngắt
ret
end main                        ; Kết thúc chương trình
```

- **XOA_MAN_HINH:** Thủ tục xóa màn hình.
- **mov ah, 3:** Dòng này di chuyển giá trị 3 vào thanh ghi ax. Trong lập trình assembly của Intel x86, thanh ghi AX thường được sử dụng để chứa các thông tin cần truyền cho các hàm hệ thống thông qua ngắt.
- **int 10h**: Đây là một ngắt tương tác với bộ điều khiển video. Trong trường hợp này, int 10h được sử dụng để gọi các dịch vụ của BIOS (Basic Input/Output System) liên quan đến hiển thị. Khi ngắt này được gọi, nó thường dựa vào giá trị trong thanh ghi ah để xác định chức năng cụ thể cần thực hiện. Trong trường hợp này, việc gán giá trị 3 cho ah có thể liên quan đến việc xóa màn hình.

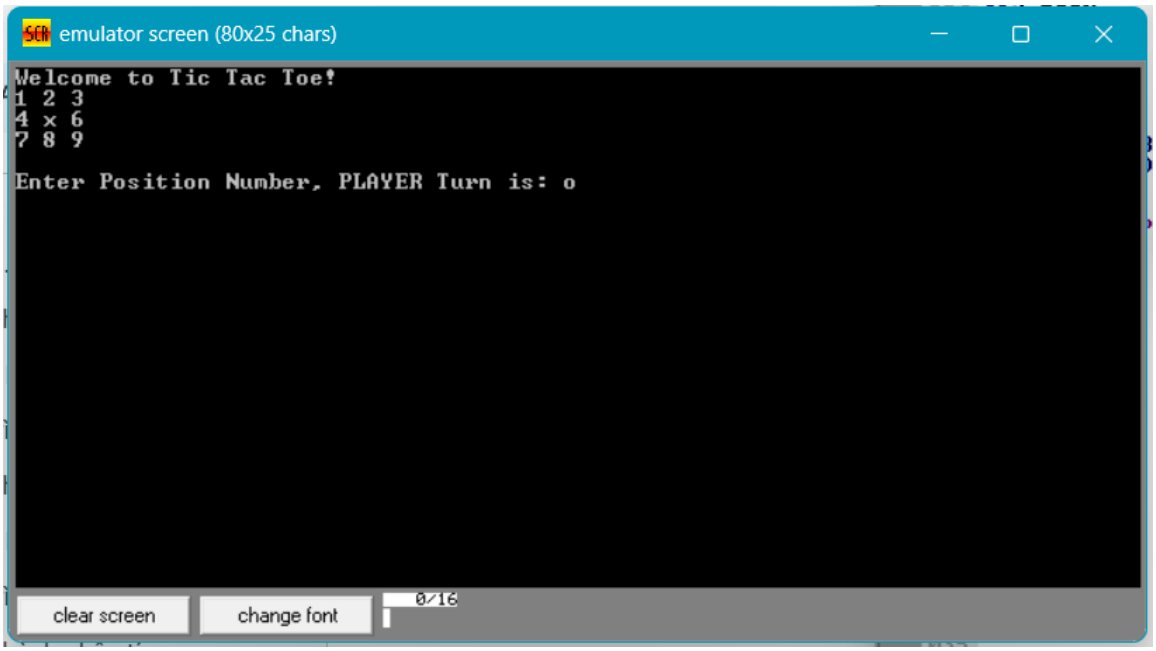
- **ret.**
- **end main**: Kết thúc chương trình.

1.4: MIÊU TẢ GIAO DIỆN CHƯƠNG TRÌNH:

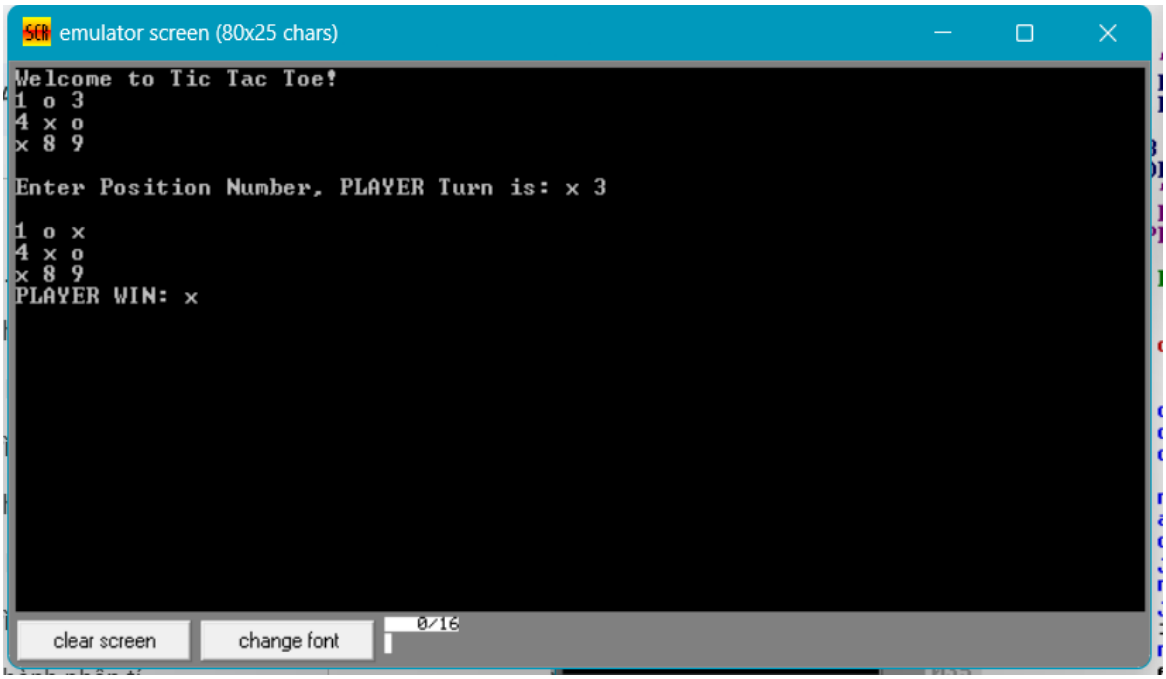
- Bắt đầu chương trình, ta sẽ có 1 ma trận kích thước 3x3 đánh số từ 1 đến 9 như sau :



- Sau đó là dòng : " **Enter position number , Player turn is** : " để chỉ định lượt chơi của người chơi X hay O.
- Sau khi nhận vị trí từ bàn phím, chương trình sẽ thực hiện in lại ma trận với dấu "**X**" hoặc "**O**" thay cho ô số nhập vào, đồng thời thông báo lượt chơi của người còn lại.
- Ví dụ số đã nhập là 5:



- Tiếp theo là các lượt chơi liên tiếp .
- Nếu 1 trong 2 người chơi thắng cuộc, màn hình sẽ in ra ma trận cùng với người giành chiến thắng :



- Nếu cả 2 người chơi hòa thì màn hình sẽ in ra ma trận cùng với câu “Draw!”.